Hannah T. Nguyen
ECE-2263 - Embedded Systems in C
Professor Friedel
December 18, 2025

Final Project

Objective

Create a working wattmeter that is able to accurately measure the power usage of a load. The wattmeter should display the voltage, current, power, and kilowatt-hours.

Introduction

A wattmeter is an important device in our society because it helps measure how much electrical power different appliances and systems use. This information allows people to monitor energy consumption and make sure devices are operating efficiently. The wattmeter works by measuring both the voltage across a circuit and the current flowing through it, then multiplying these values to determine the total power being used. In many modern wattmeters, digital sensors take these readings and provide clear, accurate results. This helps homeowners, engineers, and technicians understand their energy usage to reduce waste and improve overall electrical performance.

Equipment

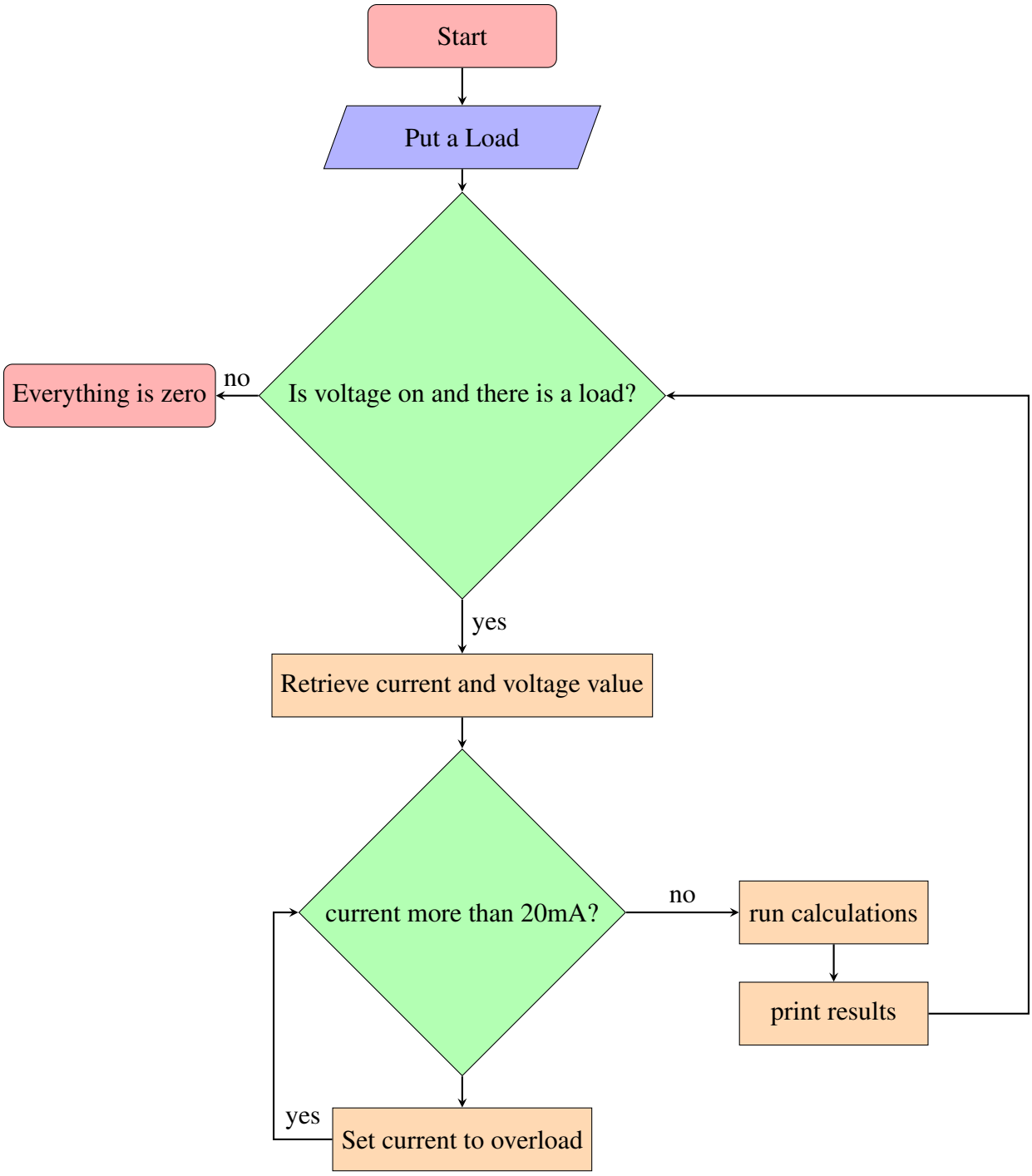
1. Arduino Uno R3
2. Arduino IDE
3. breadboard
4. jumper wires
5. op-amp
6. resistors
7. TFT LCD Screen
8. Power Supply
9. Overleaf / LaTeX

Methodology

The design included using a power supply as the main power source with the load with a small value resistor in parallel as well as two resistors in series in parallel with the power supply. The resistor in series with the load should be small enough to not affect the resistance or voltage read, but large enough for the Arduino to detect the voltage and current. The other two resistors that are in series, but in parallel with the power supply. These resistors act as a voltage divider when sensing the voltage. To calculate the resistors in series, the ratio was found and from that, the

resistors used were 10k and 1k. For the current resistor, a small value of 10 ohms was used. To have enough current for the Arduino to read it, an op-amp with a gain of 11 was placed. The full circuit is shown in figure 1. The voltage and current are the only two values that are read from the circuit. From there, the power can simply be calculated with the equation V=IR. To get the energy in kWh, the power is multiplied by the time elapsed in seconds and divided by 1000. This way, the energy is a running sum for the whole hour, no matter how long it is run for.

After initial testing, the results were off by more than 5%, so to fix it, a calibration was done for both the voltage and the current. A linear regression was used and implemented for the correction. This can be seen in figure 13. After that, the results yielded less than 5%. An example of the corrected reading is shown in figure 2. The last thing that was added was the boundaries. After testing the absolute minimum and maximum of the wattmeter, it seems that the results start to drop in quality after 20mA, so that was included in the code, which can be seen in figures 4-8.



Flowchart of the Process and Calculations

Results

As a result of this project, the custom wattmeter was created that can accurately read the voltage, current, power, and energy of a load. The error and tolerance of the wattmeter is less than 5%, which is expected and has a decent accuracy. The limits of the wattmeter is 20mA or 30 volts, whichever one hits first.

Conclusion

From this project, the wattmeter was successfully created that reads the voltage, current, power used, and kilowatts-hours. This project gave me a better understanding of wattmeters and analog-to-digital conversion.

Appendix

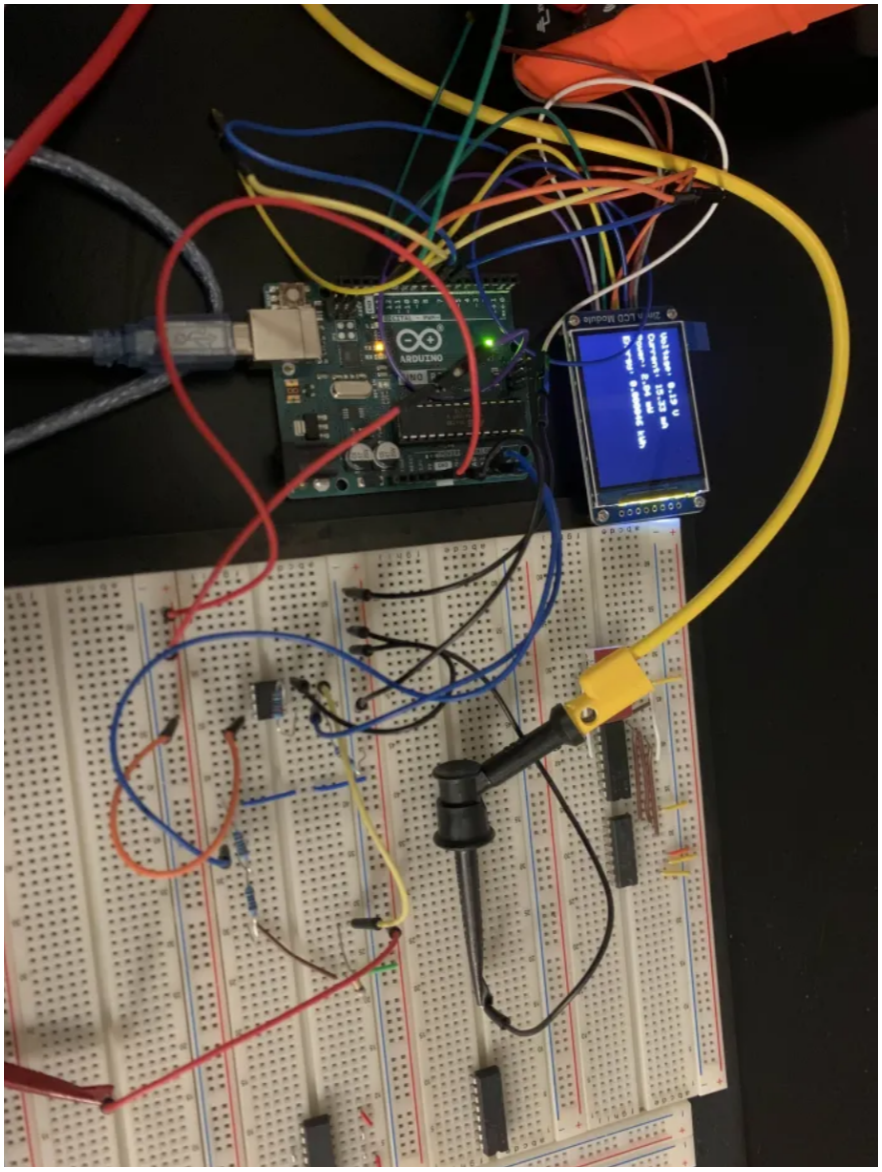


Figure 1: Complete Circuit of the Wattmeter

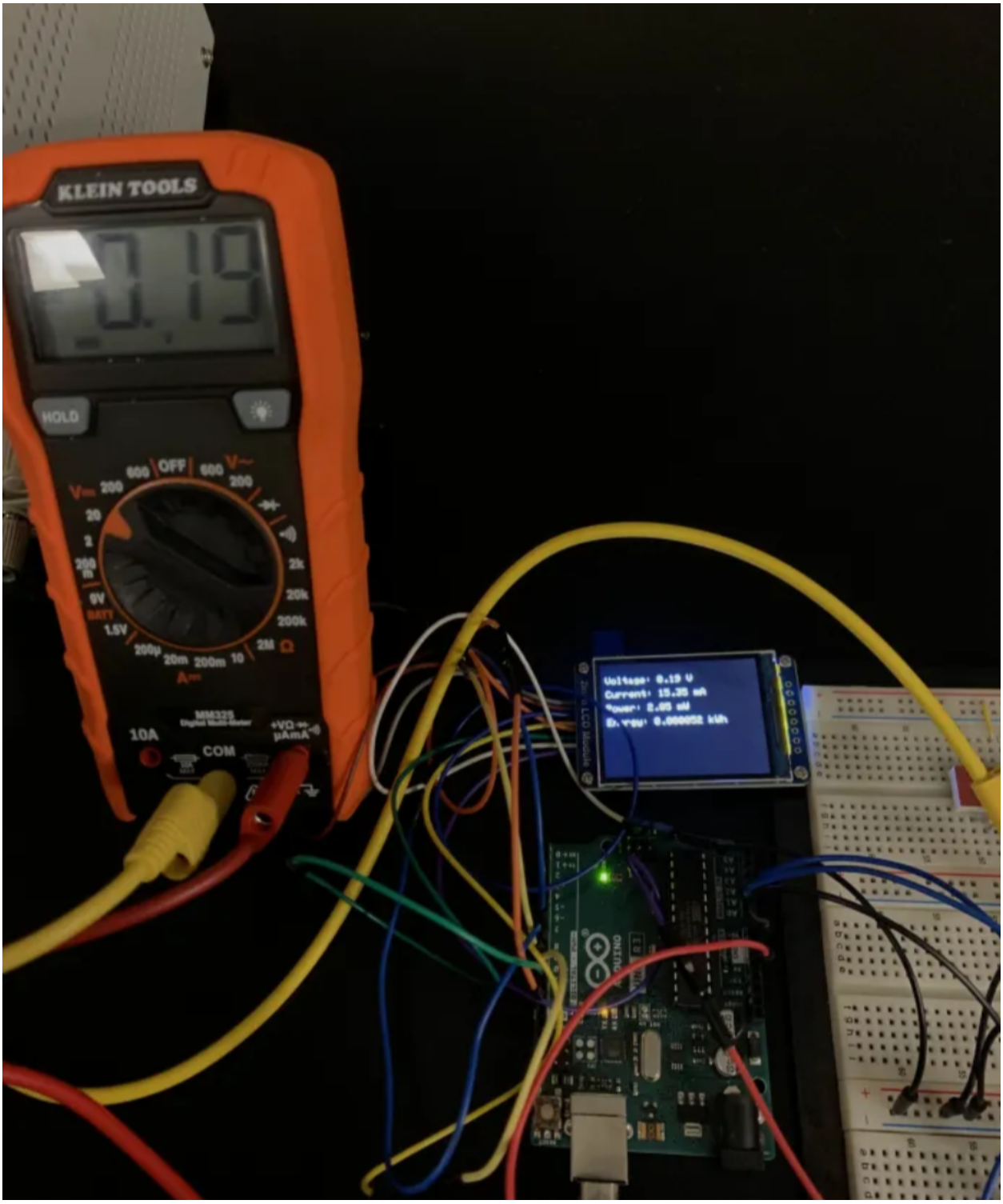


Figure 2: Multimeter reading with Screen reading

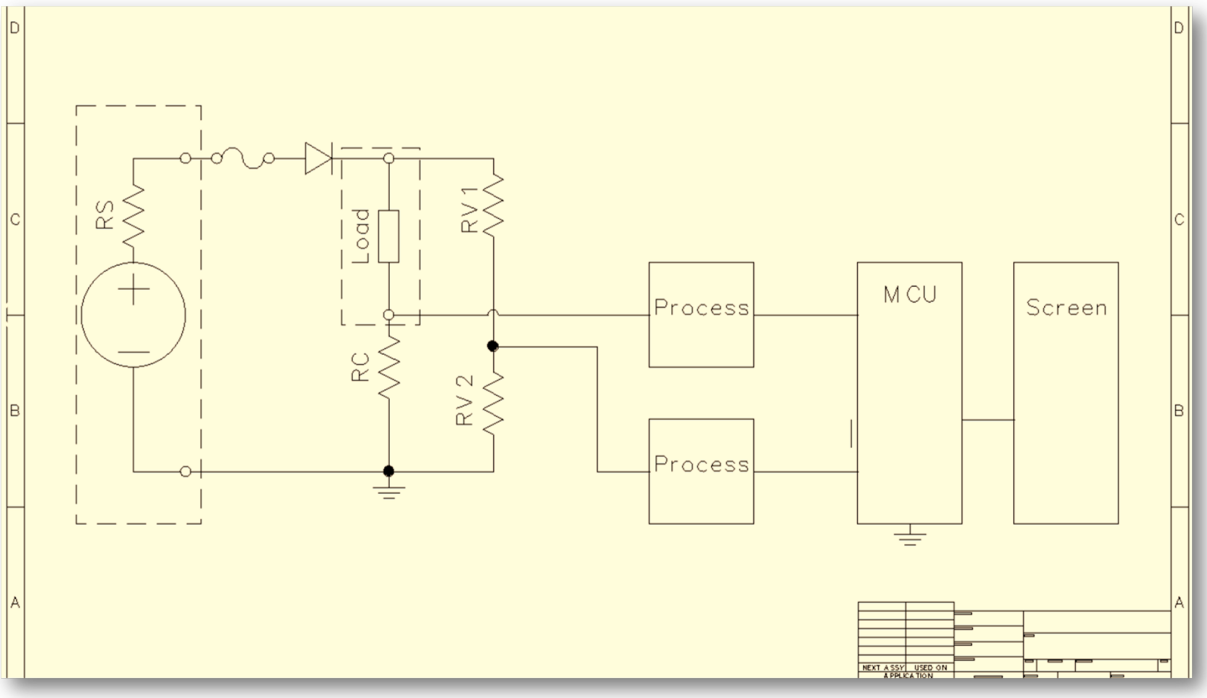


Figure 3: Schematic of the circuit

```
#include <LiquidCrystal.h>
#include <Adafruit_GFX.h>
#include <Adafruit_ST7789.h>
#include <SPI.h>

#define TFT_CS     10
#define TFT_RST    9
#define TFT_DC     8
#define TFT_BLK    7  // Backlight pin

int Read_Voltage = A0;
int Read_Current = A1;

// Constant Values
const float Rc  = 10.1;    // Current Resistor
const float RV1 = 9.91;   // top resistor of divider
const float RV2 = 0.994;    // bottom resistor of divider
// Note: RV1 an RV2 are divided so units are negligible
const float voltCorrInt = -0.138918323;//-0.118918323; // Voltage Intercept
const float voltCorrSlope = 0.992361592; // Voltage Slope Correction
const float mAmpCorrInt = -0; // Current Correction intercept in MILLIAMPS
const float mAmpCorrSlope = 0.723090313; // Current Correction slope

float Volt = 0;
float mAmp  = 0;
float kWh = 0;
float CURRENT_OFFSET = 0.0;

long time = 0;

// Values for TFT display
int x = 10;
int yVoltage = 30;
int yCurrent = 60;
int yWatt = 90;
```

Figure 4: Code Part 1

```
int ykWh = 120;
int w = 500; // width of text area
int h = 20;  // height of text area


// const int RS = 12, E = 11, D4 = 5, D5 = 4, D6 = 3, D7 = 2;
Adafruit_ST7789 tft = Adafruit_ST7789(TFT_CS, TFT_DC, TFT_RST);

void setup() {
  Serial.begin(9600);
  pinMode(TFT_BLK, OUTPUT);
  analogWrite(TFT_BLK, 255);  // Set backlight to full brightness
  time = millis();

  // Setting up TFT display
  tft.init(240, 320);
  tft.setRotation(3);  // Adjust rotation if needed
  tft.fillScreen(ST77XX_BLACK); // Black Background
  tft.setTextColor(ST77XX_WHITE); // White Text
  tft.setTextSize(2);
}

// Reading Value
float readValue(int pin) {
  // Getting the average multiple readings to reduce noise
  long sum = 0;
  for (int i = 0; i < 10; i++) {
    sum += analogRead(pin);
  }
  float avg = sum / 10.0;
  return avg;
}
```

Figure 5: Code Part 2

```
70  void loop() {
71    long startTime = millis();
72    float elapsedTime = (startTime - time) / 3600000.0;
73    time = startTime;
74
75    // Read raw ADC values (0-1023)
76    analogReference(DEFAULT);
77    delay(100);
78    float rawV = readValue(Read_Voltage);
79    float Vin = (rawV * 5.0) / 1023.0;
80    Volt = Vin * ((RV1 + RV2) / RV2); // Voltage divider
81
82    float rawA = 0; // Resets raw current data
83    rawA = readValue(Read_Current);
84    float Vsense = (rawA * 5.0) / 1023.0;
85    mAmp = ((Vsense / Rc) / 11.01) * 1000; // 11.01 is the gain from the op-amp
86    if (mAmp < 0) mAmp = 0;
87    Serial.println(rawV);
88    Serial.println(rawA);
89
90    // Clearing the screen for new values
91    tft.fillRect(x, yVoltage, w, h, ST77XX_BLACK);
92    tft.fillRect(x, yCurrent, w, h, ST77XX_BLACK);
93    tft.fillRect(x, yWatt, w, h, ST77XX_BLACK);
94    tft.fillRect(x, ykWh, w, h, ST77XX_BLACK);
95
96    // Printing the Voltage to screen
97    tft.setCursor(x, yVoltage);
98    tft.print("Voltage: ");
99    // Voltage Calibration
100   Volt = (Volt * voltCorrSlope) + voltCorrInt;
```

Figure 6: Code Part 3

```
102   // Boundary Checking
103   // No Voltage Source
104   if (Volt <= 0.05){
105     Volt = 0;
106   }
107   tft.print(Volt, 2);  // 2 decimal places
108   tft.print(" V");
109
110   // Serial output (for testing purposes)
111   Serial.print("Voltage: ");
112   Serial.println(Volt);
113
114   // Printing the Current to screen
115   tft.setCursor(x, yCurrent);
116   tft.print("Current: ");
117   // Current Calibration
118   mAmp = (mAmp * mAmpCorrSlope) + mAmpCorrInt;
119   // Boundary Checking
120   if (mAmp >= 20){
121     tft.print("Overload");
122   } else if (Volt == 0){
123     tft.print(0, 2);
124     tft.print(" mA");
125   } else {
126     tft.print(mAmp, 2);
127     tft.print(" mA");
128   }
129
130   // Serial output (for testing purposes)
131   Serial.print("Current: ");
132   Serial.println(mAmp);
133
```

Figure 7: Code Part 4

```
130     // Serial output (for testing purposes)
131     Serial.print("Current: ");
132     Serial.println(mAmp);
133
134     // Printing the Wattage to the screen
135     tft.setCursor(x, yWatt);
136     tft.print("Power: ");
137     tft.print(mAmp * Volt, 2);
138     tft.print(" mW");
139
140     // Print energy in kWh to the screen
141     tft.setCursor(x, ykWh);
142     tft.print("Energy: ");
143     kWh += (mAmp * Volt * elapsedTime) / 1000;
144     tft.print(kWh, 6);
145     tft.print(" kWh");
146
147     Serial.println();
148     delay(1000);
149 }
150
```

Figure 8: Code Part 5

Current Provided	Current Read	Current Corrected	Pct Error of Read	Pct Error of Corrected
8.95	12.56	8.964883955	40.34%	0.17%
9.41	13.15	9.39150724	39.74%	-0.20%
13.72	19.14	13.72281821	39.50%	0.02%
16.65	23.19	16.65133398	39.28%	0.01%
20.27	28.2	20.27401645	39.12%	0.02%
22.39	31.12	22.38544016	38.99%	-0.02%

Intercept (mA)	Slope
-0.117130375	0.72309031

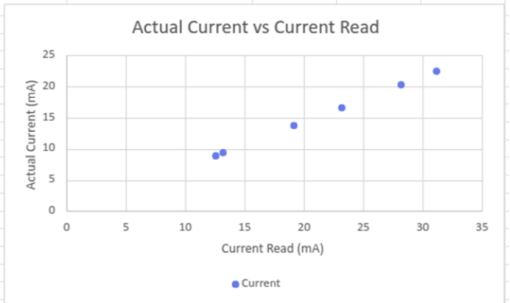


Figure 9: Current Calibration

Voltage Provided	Voltage Reac	Voltage Corrected	Pct Error of Read	Pct Error of Corrected
0.88	1.02	0.893290501	15.91%	1.51%
2.9	3.06	2.917708149	5.52%	0.61%
4.24	4.4	4.247472683	3.77%	0.18%
5.35	5.52	5.358917667	3.18%	0.17%
6.17	6.33	6.162730556	2.59%	-0.12%
7.43	7.61	7.432953395	2.42%	0.04%
8.34	8.52	8.336002444	2.16%	-0.05%
10.23	10.41	10.21156585	1.76%	-0.18%
12.15	12.35	12.13674734	1.65%	-0.11%
13.44	13.66	13.43674103	1.64%	-0.02%
15.5	15.71	15.47108229	1.35%	-0.19%
16.67	16.89	16.64206897	1.32%	-0.17%
17.92	18.16	17.90236819	1.34%	-0.10%
20.2	20.5	20.22449432	1.49%	0.12%
22.6	22.94	22.6458566	1.50%	0.20%

Intercept	Slope
-0.11891832	0.99236159

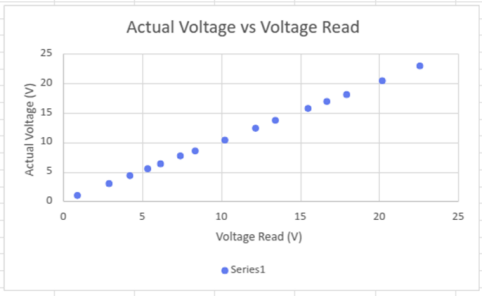
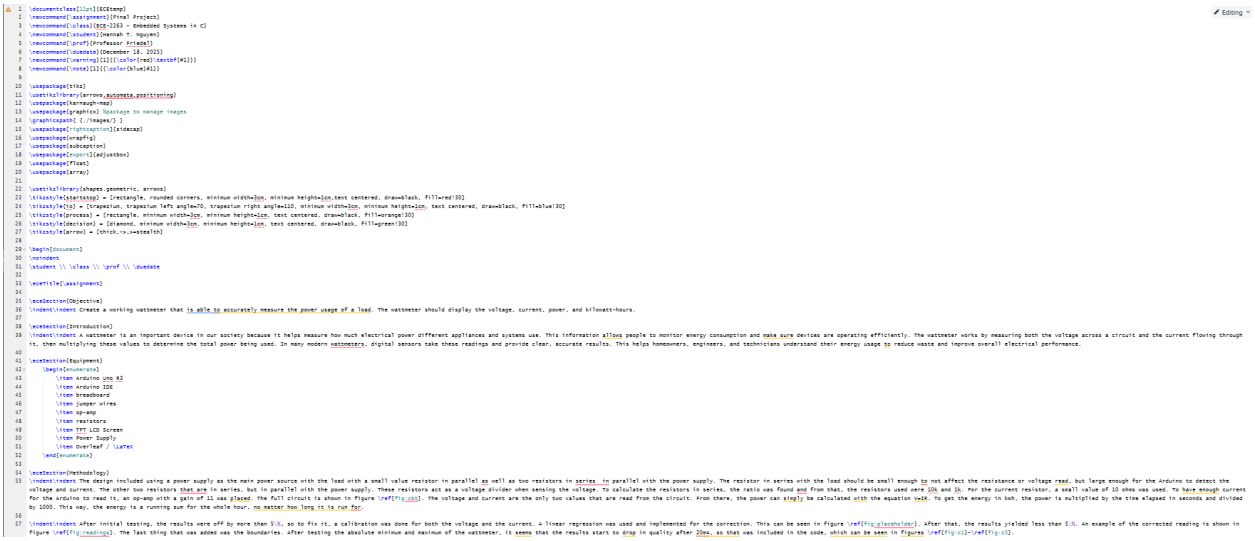


Figure 10: Voltage Calibration



```
\documentclass[11pt]{ECEtemp}
\newcommand{\assignment}{Final Project}
\newcommand{\class}{ECE-2263 - Embedded Systems in C}
\newcommand{\student}{Hannah T. Nguyen}
\newcommand{\prof}{Professor Fricke}
\newcommand{\duedate}{December 18, 2023}
\newcommand{\warning}[1]{{\color{red}\textbf{#1}}}
\newcommand{\note}[1]{{\color{blue}#1}}

\usepackage{tikz}
\usetikzlibrary{arrows,automata,positioning}
\usepackage{karnaugh-map}
\usepackage{graphicx} %package to manage images
\graphicspath{ {./images/} }
\usepackage[rightcaption]{sidecap}
\usepackage{wrapfig}
\usepackage{subcaption}
\usepackage[export]{adjustbox}
\usepackage{float}
\usepackage{array}

\usetikzlibrary{shapes.geometric, arrows}
\tikzstyle{startstop} = [rectangle, rounded corners, minimum width=3cm, minimum height=1cm,text centered, draw=black, fill=red!30]
\tikzstyle{io} = [trapezium, trapezium left angle=70, trapezium right angle=110, minimum width=3cm, minimum height=1cm, text centered, draw=black, fill=blue!30]
\tikzstyle{process} = [rectangle, minimum width=3cm, minimum height=1cm, text centered, draw=black, fill=orange!30]
\tikzstyle{decision} = [diamond, minimum width=3cm, minimum height=1cm, text centered, draw=black, fill=green!30]
\tikzstyle{arrow} = [thick,->,>=stealth]

\begin{document}
\noindent
\student \\ \class \\ \prof \\ \duedate

\ecetitle[\assignment]

\eceSection{Objective}
\indent\indent Create a working wattmeter that is able to accurately measure the power usage of a load. The wattmeter should display the voltage, current, power, and kilowatt-hours.

\ecesection{Introduction}
\indent\indent A wattmeter is an important device in our society because it helps measure how much electrical power different appliances and systems use. This information allows people to monitor energy consumption and make sure devices are operating efficiently. The wattmeter works by measuring both the voltage across a circuit and the current flowing through it, then multiplying these values to determine the total power being used. In many modern wattmeters, digital sensors take these readings and provide clear, accurate results. This helps homeowners, engineers, and technicians understand their energy usage to reduce waste and improve overall electrical performance.

\eceSection{Equipment}
  \begin{enumerate}
      \item Arduino uno R3
      \item Arduino IDE
      \item breadboard
      \item jumper wires
      \item op-amp
      \item resistors
      \item TFT LCD Screen
      \item Power Supply
      \item Overleaf / LaTeX
  \end{enumerate}

\eceSection{Methodology}
\indent\indent The design included using a power supply as the main power source with the load with a small value resistor in parallel as well as two resistors in series  in parallel with the power supply. The resistor in series with the load should be small enough to not affect the resistance or voltage read, but large enough for the Arduino to detect the voltage and current. The other two resistors that are in series, but in parallel with the power supply. These resistors act as a voltage divider when sensing the voltage. To calculate the resistors in series, the ratio was found and from that, the resistors used were 20k and 1k. For the current resistor, a small value of 10 ohms was used. To have enough current for the Arduino to read it, an op-amp with a gain of 11 was placed. The full circuit is shown in figure \ref{fig:ckt1}. The voltage and current are the only two values that are read from the circuit. From there, the power can simply be calculated with the equation v*ik. To get the energy in kwh, the power is multiplied by the time elapsed in seconds and divided by 1000. This way, the energy is a running sum for the whole hour, no matter how long it is run for.

\indent\indent After initial testing, the results were off by more than 5%, so to fix it, a calibration was done for both the voltage and the current. A linear regression was used and implemented for the correction. This can be seen in figure \ref{fig:placeholder}. After that, the results yielded less than 5%. An example of the corrected reading is shown in figure \ref{fig:readings}. The last thing that was added was the boundaries. After testing the absolute minimum and maximum of the wattmeter, it seems that the results start to drop in quality after 20ma, so that was included in the code, which can be seen in figures \ref{fig:ol1-\ref{fig:ol2}.
```

Figure 11: LaTeXPart 1

```
\begin{center}
    \begin{tikzpicture}[node distance=1.61cm]
    <TikZ code>
    \node (start) [startstop] {Start};
    \node (input) [io, below of=start] {Put a Load};
    \node (checkRange) [decision, below of=input, yshift=-2.5cm] {Is voltage on and there is a load?};
    \node (2meters) [process, below of=checkRange, yshift=-3cm]{Retrieve current and voltage value};
    \node (stop) [startstop, left of=checkRange, xshift=-4cm] {Everything is zero};
    \node (loop) [decision, below of=2meters, yshift=-2cm]{current more than 20mA?};
    \node (temps) [process, below of=loop, yshift=-2cm]{Set current to overload};


    \node (continue) [process, right of=loop, xshift=4.3cm]{run calculations};
    \node (time) [process, below of=continue]{print results};

    \draw [arrow] (start) -- (input);
    \draw [arrow] (input) -- (checkRange);
    \draw [arrow] (checkRange) -- node[anchor=south]{no}(stop);
    \draw [arrow] (checkRange) -- node[anchor=west]{yes}(2meters);
    \draw [arrow] (2meters) -- (loop);
    \draw [arrow] (loop) -- (temps);
    \draw [arrow] (temps) -- node[anchor=south] {yes} ++(-1.-2,0) -- ++(0,1) |- (loop);

    \draw [arrow] (loop) -- node[anchor=south]{no}(continue);
    \draw [arrow] (continue) -- (time);
    \draw [arrow] (time) -- node[anchor=north] {} ++(3.1,0) -- ++(0,1) |-(checkRange);
    \end{tikzpicture}\\
    Flowchart of the Process and Calculations
\end{center}

\secSection{Results}
\indent\indent As a result of this project, the custom wattmeter was created that can accurately read the voltage, current, power, and energy of a load. The error and tolerance of the wattmeter is less than 5\%, which is expected and has a decent accuracy. The limits of the wattmeter is 20mA or 30 volts, whichever one hits first.

\secSection{Conclusion}
\indent\indent From this project, the wattmeter was successfully created that reads the voltage, current, power used, and kilowatts-hours. This project gave me a better understanding of wattmeters and analog-to-digital conversion.

\secSection{Appendix}
\begin{figure}[H]
    \centering
    \includegraphics[width=0.7\linewidth]{images/fullckt.png}
    \caption{Complete Circuit of the Wattmeter}
    \label{fig:ckt}
\end{figure}

\begin{figure}[H]
    \centering
    \includegraphics[width=1\linewidth]{images/readings.png}
    \caption{Multimeter reading with Screen reading}
    \label{fig:readings}
\end{figure}

\begin{figure}[H]
    \centering
    \includegraphics[width=1\linewidth]{images/schematic.png}
    \caption{Schematic of the circuit}
    \label{fig:placeholder}
\end{figure}

\begin{figure}[H]
    \centering
    \includegraphics[width=0.8\linewidth]{images/code1.png}
    \caption{Code Part 1}
    \label{fig:c1}
\end{figure}
```

Figure 12: LaTeXPart 2

```
\begin{figure}[H]
    \centering
    \includegraphics[width=0.8\linewidth]{images/Code2.png}
    \caption{Code Part 2}
    \label{fig:c2}
\end{figure}

\begin{figure}[H]
    \centering
    \includegraphics[width=1\linewidth]{images/Code3.png}
    \caption{Code Part 3}
    \label{fig:c3}
\end{figure}

\begin{figure}[H]
    \centering
    \includegraphics[width=0.8\linewidth]{images/Code4.png}
    \caption{Code Part 4}
    \label{fig:c4}
\end{figure}

\begin{figure}[H]
    \centering
    \includegraphics[width=0.8\linewidth]{images/Code5.png}
    \caption{Code Part 5}
    \label{fig:c5}
\end{figure}

\begin{figure}[H]
    \centering
    \includegraphics[width=1\linewidth]{images/currCal.png}
    \caption{Current Calibration}
    \label{fig:placeholder}
\end{figure}

\begin{figure}[H]
    \centering
    \includegraphics[width=1\linewidth]{images/voltCal.png}
    \caption{Voltage Calibration}
    \label{fig:placeholder}
\end{figure}

\begin{figure}[H]
    \centering
    \includegraphics[width=1\linewidth]{images/latex1.png}
    \caption{LaTeX Part 1}
    \label{fig:placeholder}
\end{figure}

\begin{figure}[H]
    \centering
    \includegraphics[width=1\linewidth]{images/latex2.png}
    \caption{LaTeX Part 2}
    \label{fig:placeholder}
\end{figure}

\begin{figure}[H]
    \centering
    \includegraphics[width=1\linewidth]{images/latex3.png}
    \caption{LaTeX Part 3}
    \label{fig:placeholder}
\end{figure}

\end{document}
```

Figure 13: LaTeXPart 3